

Decentralized EHR System — Multi-System Setup Guide

| Hyperledger Fabric 2.5 + IPFS | 3 Organizations · 5 Peers · 3 Physical Machines

Table of Contents

1. [Architecture Overview](#)
 2. [What Runs Where](#)
 3. [Prerequisites](#)
 4. [Phase 0 — Configure hosts.env](#)
 5. [Phase 1 — Network Connectivity & Firewall](#)
 6. [Phase 2 — SSH Passwordless Access \(SYS1 only\)](#)
 7. [Phase 3 — Start CAs and Enroll Identities \(SYS1\)](#)
 8. [Phase 4 — Distribute Files to SYS2 and SYS3](#)
 9. [Phase 5 — Setup SYS2 \(peer0.hospital\)](#)
 10. [Phase 6 — Setup SYS3 \(peer2.hospital\)](#)
 11. [Phase 7 — Start SYS1 Network Containers](#)
 12. [Phase 8 — Create Channel and Join All Peers](#)
 13. [Phase 9 — Set Anchor Peers](#)
 14. [Phase 10 — Deploy Chaincode](#)
 15. [Phase 11 — Start Backend Services](#)
 16. [Phase 12 — Start Frontend Portals](#)
 17. [Phase 13 — Verification](#)
 18. [Environment Variable Reference](#)
 19. [Port Reference](#)
 20. [Troubleshooting](#)
 21. [Tear Down and Clean Up](#)
-

1. Architecture Overview

ehrchannel (shared ledger)
All 3 orgs hold a full ledger replica

SYS1		SYS2	SYS3
orderer	:7050	peer0.hospital	peer2.hospital
peer1.hospital	:9051	:7051	:10051
peer0.diagnostic	:8051		
peer0.provider	:11051	peer0-api	peer2-api
ca-orderer	:7054	:3001	:3003
ca-hospital	:7055		
ca-diagnostics	:7056		
ca-provider	:7057		
IPFS (Kubo)	:5001/8090		
ipfs-service	:3006		
peer1-api	:3002		
extorg-api	:3004		
patient-api	:3005		

Organizations and peer-to-role mapping:

Org	MSP ID	Peer	Roles
Hospital	HospitalMSP	peer0 (SYS2, :7051)	Receptionist, Admin
Hospital	HospitalMSP	peer1 (SYS1, :9051)	Doctor
Hospital	HospitalMSP	peer2 (SYS3, :10051)	Nurse, Pharmacist, MedRecordOfficer
Diagnostics	DiagnosticsMSP	peer0.diagnostic (SYS1, :8051)	Lab roles
Provider	ProviderMSP	peer0.provider (SYS1, :11051)	Billing, Insurance

2. What Runs Where

Component	System	Port	Notes
orderer	SYS1	7050	Raft CFT consensus
peer0.hospital	SYS2	7051	Auth/Reception peer
peer1.hospital	SYS1	9051	Doctor peer
peer2.hospital	SYS3	10051	Nurse/Pharmacist peer
peer0.diagnostic	SYS1	8051	Lab peer
peer0.provider	SYS1	11051	Insurance peer
ca-orderer	SYS1	7054	
ca-hospital	SYS1	7055	
ca-diagnostics	SYS1	7056	
ca-provider	SYS1	7057	
IPFS (Kubo)	SYS1	5001 / 8090	API / Gateway
ipfs-service	SYS1	3006	Internal IPFS wrapper
peer0-api	SYS2	3001	Receptionist + Admin backend
peer1-api	SYS1	3002	Doctor backend
peer2-api	SYS3	3003	Nurse / Pharmacist backend
extorg-api	SYS1	3004	Lab + Provider backend
patient-api	SYS1	3005	Patient portal backend
hospital-portal	any	5173	React frontend (staff)
patient-portal	any	5174	React frontend (patients)

3. Prerequisites

Install the following on **all three systems** before starting.

3.1 Docker and Docker Compose

```
bash

sudo apt update
sudo apt install -y docker.io docker-compose-plugin curl jq
sudo usermod -aG docker $USER
newgrp docker
docker run hello-world    # verify – should print "Hello from Docker"
```

3.2 Node.js 18+ (all systems)

```
bash

curl -fsSL https://deb.nodesource.com/setup_18.x | sudo -E bash -
sudo apt install -y nodejs
node --version    # must be >= 18
npm --version
```

3.3 Hyperledger Fabric Binaries (SYS1 only)

The `peer`, `osnadmin`, `configtxgen`, and `fabric-ca-client` binaries are only needed on SYS1. They are used to create the channel, enroll identities, and deploy chaincode.

```
bash

# On SYS1
curl -sSL https://bit.ly/2ysb0FE | bash -s -- 2.5.0 1.5.7
# This installs binaries to ~/fabric-samples/bin

# Add to ~/.bashrc (replace path if different)
echo 'export PATH=$PATH:$HOME/fabric-samples/bin' >> ~/.bashrc
echo 'export FABRIC_CFG_PATH=$HOME/fabric-samples/config' >> ~/.bashrc
source ~/.bashrc

# Verify
peer version
fabric-ca-client version
configtxgen --version
```

3.4 Get Each System's IP

Run this on each machine and write the IPs down — you'll need them throughout:

```
bash

hostname -I | awk '{print $1}'
```

4. Phase 0 — Configure `hosts.env`

`hosts.env` is the **single source of truth** for all IP addresses and usernames. Edit it once and every script will pick it up automatically.

On **SYS1**, open `ehr-network/hosts.env`:

```
bash
nano ehr-network/hosts.env
```

Fill in the real values:

```
bash

# System 1: orderer, peer1, diagnostic, provider, CAs, IPFS, backends
SYS1_IP=<YOUR_SYS1_IP>

# System 2: peer0.hospital (Auth/Reception)
SYS2_IP=<YOUR_SYS2_IP>
SYS2_USER=<SYS2_USERNAME>      # e.g. ubuntu, jenish-shiroya

# System 3: peer2.hospital (Nurse/Pharmacist)
SYS3_IP=<YOUR_SYS3_IP>
SYS3_USER=<SYS3_USERNAME>      # e.g. ubuntu, rahul

# Remote base paths (where files land on SYS2 and SYS3)
SYS2_NETWORK_DIR=/home/${SYS2_USER}/Data/fabric-samples/multi-system-test/ehr-network
SYS2_BACKEND_DIR=/home/${SYS2_USER}/Data/fabric-samples/multi-system-test/ehr-backend

SYS3_NETWORK_DIR=/home/${SYS3_USER}/Data/fabric-samples/multi-system-test/ehr-network
SYS3_BACKEND_DIR=/home/${SYS3_USER}/Data/fabric-samples/multi-system-test/ehr-backend
```

Tip: All scripts in `ehr-network/scripts/` source this file automatically at startup via `scripts/envVar.sh`. You never need to export these manually.

5. Phase 1 — Network Connectivity & Firewall

Fabric peers communicate directly over TCP. These ports must be reachable across machines.

On **SYS1** — allow incoming from **SYS2** and **SYS3**

```
bash
```

```
sudo ufw allow from <SYS2_IP> to any port 7050 # orderer gRPC
sudo ufw allow from <SYS2_IP> to any port 7053 # orderer admin (osnadmin)
sudo ufw allow from <SYS2_IP> to any port 9051 # peer1 gRPC
sudo ufw allow from <SYS2_IP> to any port 8051 # diagnostic peer
sudo ufw allow from <SYS2_IP> to any port 11051 # provider peer
sudo ufw allow from <SYS2_IP> to any port 3006 # ipfs-service

sudo ufw allow from <SYS3_IP> to any port 7050
sudo ufw allow from <SYS3_IP> to any port 7053
sudo ufw allow from <SYS3_IP> to any port 9051
sudo ufw allow from <SYS3_IP> to any port 3006
```

On SYS2 — allow incoming from SYS1 and SYS3

```
bash
```

```
sudo ufw allow from <SYS1_IP> to any port 7051 # peer0 gRPC
sudo ufw allow from <SYS1_IP> to any port 7052 # peer0 chaincode listener
sudo ufw allow from <SYS3_IP> to any port 7051
sudo ufw allow from <SYS3_IP> to any port 7052
sudo ufw allow from <SYS1_IP> to any port 3001 # peer0-api
```

On SYS3 — allow incoming from SYS1 and SYS2

```
bash
```

```
sudo ufw allow from <SYS1_IP> to any port 10051 # peer2 gRPC
sudo ufw allow from <SYS1_IP> to any port 10052 # peer2 chaincode listener
sudo ufw allow from <SYS2_IP> to any port 10051
sudo ufw allow from <SYS2_IP> to any port 10052
sudo ufw allow from <SYS1_IP> to any port 3003 # peer2-api
```

Test connectivity

```
bash
```

```
# From SYS1 — both should print "succeeded"
nc -zv <SYS2_IP> 22
nc -zv <SYS3_IP> 22
```

6. Phase 2 — SSH Passwordless Access (SYS1 only)

The `distribute.sh` script SSHes from SYS1 into SYS2 and SYS3 to push files. Set up passwordless SSH first.

```
bash

# On SYS1
ls ~/.ssh/id_ed25519 2>/dev/null || ssh-keygen -t ed25519 -N "" -f ~/.ssh/id_ed25519

ssh-copy-id <SYS2_USER>@<SYS2_IP>
ssh-copy-id <SYS3_USER>@<SYS3_IP>

# Test – both should print without asking for a password
ssh <SYS2_USER>@<SYS2_IP> "echo ✓ connected to SYS2"
ssh <SYS3_USER>@<SYS3_IP> "echo ✓ connected to SYS3"
```

7. Phase 3 — Start CAs and Enroll Identities (SYS1)

All crypto material is generated on SYS1. Remote systems receive a copy later.

```
bash

# On SYS1
cd ehr-network

# Add Fabric binaries to PATH (if not already in .bashrc)
export PATH=$PATH:$HOME/fabric-samples/bin
export FABRIC_CFG_PATH=$HOME/fabric-samples/config

# Step 1: Create the Docker network
docker network create ehr_network

# Step 2: Start all four CAs
docker compose -f docker/docker-compose-ca.yaml up -d

# Step 3: Wait for CA cert files to appear (usually ~10 seconds)
sleep 15

# Verify all four CA certs were generated
ls organizations/ordererOrganizations/example.com/ca/ca-cert.pem
ls organizations/peerOrganizations/hospital.example.com/ca/ca-cert.pem
ls organizations/peerOrganizations/diagnostic.example.com/ca/ca-cert.pem
ls organizations/peerOrganizations/provider.example.com/ca/ca-cert.pem
```

If any cert file is missing, wait another 10 seconds and check again. If still missing, check `docker logs ca-hospital` for errors.

```
bash
```

```
# Step 4: Fix ownership (CA containers write as root)
```

```
sudo chown -R $USER:$USER organizations/
```

```
# Step 5: Enroll all identities (orderer, all hospital/diagnostic/provider roles)
```

```
bash scripts/enroll.sh
```

This registers and enrolls: orderer admin, hospital admin and all roles (receptionist, doctor, nurse, pharmacist, medrecordofficer, patientService), diagnostic admin and all lab roles, provider admin and all provider roles. Each identity gets a role attribute embedded in its X.509 certificate (`role=doctor:ecert`, `role=nurse:ecert`, etc.).

Expected output (last few lines):

```
✓ Enrolled: patientService
✓ Enrolled: labtechnician
✓ Enrolled: insuranceofficer
[OK] All identities enrolled
```

8. Phase 4 — Distribute Files to SYS2 and SYS3

Now that `organizations/` is fully populated with all crypto material, push everything to the remote systems in one command.

```
bash
```

```
# On SYS1
```

```
cd ehr-network
```

```
bash scripts/distribute.sh
```

This script reads `hosts.env` automatically and does the following for each remote system:

- Creates the target directories on the remote machine
- Copies `organizations/` (all MSP and TLS certs for all orgs)
- Copies `configtx/core.yaml` (peer configuration)
- Copies the correct `docker/docker-compose-sys2.yaml` or `docker-compose-sys3.yaml`
- Copies the corresponding backend service code (`peer0-api/` or `peer2-api/`)
- Pre-populates the backend `.env` with the correct `FABRIC_BASE_PATH` and `IPFS_SERVICE_URL=http://<SYS1_IP>:3006`
- Configures `/etc/hosts` on each remote machine with the right hostname-to-IP mappings

Expected output:


```
→ Distributing to SYS2 (jenish-shiroya@10.162.37.24)...
  ✓ Created directories
  ✓ Copied organizations/
  ✓ Copied core.yaml
  ✓ Copied docker-compose-sys2.yaml
  ✓ Copied peer0-api/
  ✓ Configured /etc/hosts
→ Distributing to SYS3 (rahul@10.162.37.79)...
  ✓ Created directories
  ...
[DONE] Distribution complete
```

Note: The distribute script copies the **full** `organizations/` directory to remote systems (including all org MSPs). This is intentional — each peer needs the MSP certs of all three organizations to verify cross-org transactions. Private keys (`keystore/*_sk`, `server.key`) are in subdirectories that are only used by the local peer/user identity; remote systems simply ignore them.

9. Phase 5 — Setup SYS2 (peer0.hospital)

SSH into SYS2:

```
bash

ssh <SYS2_USER>@<SYS2_IP>
```

9.1 Create Docker network on SYS2

```
bash

docker network create ehr_network
```

Each system creates its own independent `ehr_network` Docker bridge. They are **not** connected at the Docker level — cross-host communication happens over real TCP ports on the host network. Docker's `extra_hosts` in the compose file injects hostname→IP mappings into each container.

9.2 Create the docker .env file

```
bash

cat > ~/Data/fabric-samples/multi-system-test/ehr-network/docker/.env <<EOF
SYS1_IP=<SYS1_IP>
SYS3_IP=<SYS3_IP>
EOF
```

9.3 Start peer0.hospital

```
bash
cd ~/Data/fabric-samples/multi-system-test/ehr-network/docker
docker compose -f docker-compose-sys2.yaml up -d
```

9.4 Verify peer0 is running

```
bash
docker ps
# Expected: peer0.hospital.example.com    Up

docker logs $(docker ps -q --filter name=peer0.hospital) --tail 20
# Expected: Starting peer with ID=[peer0.hospital.example.com:7051]
```

10. Phase 6 — Setup SYS3 (peer2.hospital)

SSH into SYS3:

```
bash
ssh <SYS3_USER>@<SYS3_IP>
```

10.1 Create Docker network on SYS3

```
bash
docker network create ehr_network
```

10.2 Create the docker .env file

```
bash
cat > ~/Data/fabric-samples/multi-system-test/ehr-network/docker/.env <<EOF
SYS1_IP=<SYS1_IP>
SYS2_IP=<SYS2_IP>
EOF
```

10.3 Start peer2.hospital

```
bash
cd ~/Data/fabric-samples/multi-system-test/ehr-network/docker
docker compose -f docker-compose-sys3.yaml up -d
```

10.4 Verify peer2 is running

```
bash
```

```
docker ps
```

```
# Expected: peer2.hospital.example.com    Up
```

```
docker logs $(docker ps -q --filter name=peer2.hospital) --tail 20
```

```
# Expected: Starting peer with ID=[peer2.hospital.example.com:10051]
```

11. Phase 7 — Start SYS1 Network Containers

Back on SYS1:

```
bash
```

```
cd ehr-network
```

```
docker compose -f docker/docker-compose-sys1.yaml up -d
```

Verify all SYS1 containers are up

```
bash
```

```
docker ps --format "table {{.Names}}\t{{.Status}}"
```

Expected containers:

NAMES	STATUS
orderer.example.com	Up
peer1.hospital.example.com	Up
peer0.diagnostic.example.com	Up
peer0.provider.example.com	Up
ipfs.ehr.local	Up (healthy)
ca-orderer	Up
ca-hospital	Up
ca-diagnostics	Up
ca-provider	Up

Verify cross-host connectivity from SYS1

```
bash
```

```
# peer0 on SYS2 must be reachable
```

```
nc -zv <SYS2_IP> 7051 # should print: succeeded
```

```
# peer2 on SYS3 must be reachable
```

```
nc -zv <SYS3_IP> 10051 # should print: succeeded
```

If either fails, recheck the firewall rules in Phase 1 and confirm the Docker containers are running on the remote machines.

12. Phase 8 — Create Channel and Join All Peers

Run entirely from SYS1. The `peer` CLI binary connects to remote peers via the hostnames injected in `/etc/hosts` (set up by `distribute.sh`).

```
bash

# On SYS1
cd ehr-network
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export TEST_NETWORK_HOME=$(pwd)

bash scripts/createChannel.sh --channel ehrchannel
```

What this does, step by step:

1. Runs `configtxgen` to produce `channel-artifacts/ehrchannel.block`
2. Posts the genesis block to the orderer via `osnadmin channel join` (mutual TLS)
3. Joins `peer0.hospital` (SYS2) — CLI connects to `peer0.hospital.example.com:7051`
4. Joins `peer1.hospital` (SYS1) — CLI connects to `localhost:9051`
5. Joins `peer2.hospital` (SYS3) — CLI connects to `peer2.hospital.example.com:10051`
6. Joins `peer0.diagnostic` (SYS1) — CLI connects to `localhost:8051`
7. Joins `peer0.provider` (SYS1) — CLI connects to `localhost:11051`

Expected output (last few lines):

```
✓ peer0.hospital joined ehrchannel
✓ peer1.hospital joined ehrchannel
✓ peer2.hospital joined ehrchannel
✓ peer0.diagnostic joined ehrchannel
✓ peer0.provider joined ehrchannel
[OK] Channel ehrchannel created. All 5 peers joined.
```

If a peer join fails, check its container logs:

```
bash

# peer0 on SYS2
ssh <SYS2_USER>@<SYS2_IP> "docker logs \$(docker ps -q --filter name=peer0.hospital)

# peer2 on SYS3
ssh <SYS3_USER>@<SYS3_IP> "docker logs \$(docker ps -q --filter name=peer2.hospital)
```

13. Phase 9 — Set Anchor Peers

Anchor peers enable cross-org gossip discovery. Without them peers in different organizations cannot discover each other. Run from SYS1:

```
bash

cd ehr-network
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
export TEST_NETWORK_HOME=$(pwd)

bash scripts/setAnchorPeer.sh 1 ehrchannel # Hospital anchor = peer0.hospital (SYS
bash scripts/setAnchorPeer.sh 2 ehrchannel # Diagnostics anchor = peer0.diagnostics
bash scripts/setAnchorPeer.sh 3 ehrchannel # Provider anchor = peer0.provider (SYS
```

Verify anchor peers were set:

```
bash

peer channel fetch config /tmp/config.pb \
  -o localhost:7050 \
  --ordererTLSTLSHostnameOverride orderer.example.com \
  -c ehrchannel --tls \
  --cafile organizations/ordererOrganizations/example.com/orderers/orderer.example.c

configtxlator proto_decode --input /tmp/config.pb \
  --type common.Block --output /tmp/config.json

jq '.data.data[0].payload.data.config.channel_group.groups.Application.groups |
  to_entries[] | {org: .key,
    anchor: .value.values.AnchorPeers.value.anchor_peers[0]}' /tmp/config.json
```

Expected output:

```
json

{"org": "HospitalMSP", "anchor": {"host": "peer0.hospital.example.com", "port":
{"org": "DiagnosticsMSP", "anchor": {"host": "peer0.diagnostics.example.com", "port":
{"org": "ProviderMSP", "anchor": {"host": "peer0.provider.example.com", "port":
```

14. Phase 10 — Deploy Chaincode

Run from SYS1. The script installs on all 5 peers, approves for all 3 orgs, and commits to the channel:

```
bash
cd ehr-network

bash scripts/deployChaincode.sh \
  --channel ehrchannel \
  --chaincode-name ehr \
  --chaincode-path ../ehr-chaincode-v3 \
  --version 1.0 \
  --sequence 1
```

What happens internally:

Step	Action
1	<code>npm install</code> in <code>ehr-chaincode-v3/</code>
2	<code>peer lifecycle chaincode package ehr_1.0.tar.gz</code>
3	Install on peer0.hospital (SYS2, remote)
4	Install on peer0.diagnostic (SYS1, local)
5	Install on peer0.provider (SYS1, local)
6	Install on peer1.hospital (SYS1, local)
7	Install on peer2.hospital (SYS3, remote)
8	<code>approveformyorg</code> for HospitalMSP, DiagnosticsMSP, ProviderMSP
9	<code>checkcommitreadiness</code> — all 3 orgs must show <code>true</code>
10	<code>chaincode commit</code> on the channel
11	Verify committed chaincode

Expected final output:

```
Committed chaincode definition for chaincode 'ehr' on channel 'ehrchannel':
Version: 1.0, Sequence: 1, ...
[OK] Chaincode deployed successfully
```

Note: On the first transaction, each peer downloads the chaincode Docker image. SYS2 and SYS3 must have internet access for this to work.

15. Phase 11 — Start Backend Services

15.1 SYS2 — peer0-api (Receptionist + Admin)

```
bash

ssh <SYS2_USER>@<SYS2_IP>
cd ~/Data/fabric-samples/multi-system-test/ehr-backend/peer0-api

# Edit .env — set your actual paths
nano .env
```

Verify these two lines are correct:

```
bash

FABRIC_BASE_PATH=/home/<SYS2_USER>/Data/fabric-samples/multi-system-test/ehr-network
IPFS_SERVICE_URL=http://<SYS1_IP>:3006
```

```
bash

npm install
npm start
# Expected:
# Server running on port 3001
# Gateway ready: receptionist
# Gateway ready: admin
```

15.2 SYS3 — peer2-api (Nurse / Pharmacist / MedRecordOfficer)

```
bash

ssh <SYS3_USER>@<SYS3_IP>
cd ~/Data/fabric-samples/multi-system-test/ehr-backend/peer2-api

nano .env
```

Verify:

```
bash

FABRIC_BASE_PATH=/home/<SYS3_USER>/Data/fabric-samples/multi-system-test/ehr-network
IPFS_SERVICE_URL=http://<SYS1_IP>:3006
```

```
bash

npm install
npm start
# Expected:
# Server running on port 3003
# Gateway ready: nurse
# Gateway ready: pharmacist
# Gateway ready: medrecordofficer
```

15.3 SYS1 — All Remaining Backends

Run each in a separate terminal (or use `tmux` / `screen`):

Terminal 1 — IPFS Service

```
bash

cd ehr-backend-v3/ipfs-service
cp .env.example .env
# Edit .env: IPFS_API_URL=http://localhost:5001
npm install
npm start
# Expected: IPFS service listening on port 3006
```

Terminal 2 — peer1-api (Doctor)

```
bash

cd ehr-backend-v3/peer1-api
cp .env.example .env
nano .env
# Set: FABRIC_BASE_PATH=<absolute path to ehr-network>
# Set: IPFS_SERVICE_URL=http://localhost:3006
npm install
npm start
# Expected: Server running on port 3002 | Gateway ready: doctor
```

Terminal 3 — extorg-api (Lab + Provider)

```
bash

cd ehr-backend-v3/extorg-api
cp .env.example .env
nano .env
# Set: FABRIC_BASE_PATH=<absolute path to ehr-network>
# Set: IPFS_SERVICE_URL=http://localhost:3006
npm install
npm start
# Expected: Server running on port 3004 | Gateway ready: labreceptionist ...
```


Terminal 4 — patient-api (Patient Portal)

```
bash

cd ehr-backend-v3/patient-api
cp .env.example .env
nano .env
# Set: FABRIC_BASE_PATH=<absolute path to ehr-network>
# Set: IPFS_SERVICE_URL=http://localhost:3006
npm install
npm start
# Expected: Server running on port 3005 | SQLite ready | Gateway ready: patientServi
```

Quick start script (SYS1)

If you want to start all SYS1 backends in one go:

```
bash

cd ehr-backend-v3
bash start_all.sh
```

16. Phase 12 — Start Frontend Portals

The frontends can run on any machine that has network access to the backend APIs. Typically run on SYS1 or on your laptop.

Hospital Portal

```
bash

cd ehr-frontend-v2/hospital-portal
npm install
npm run dev
# Listening on: http://localhost:5173
```

Set the API base URL. In `src/utils/api.js`, the `MGMT_URL` points to peer0-api on SYS2. Update if your IPs differ:

```
javascript

const MGMT_URL = 'http://<SYS2_IP>:3001'
```

Patient Portal

```
bash

cd ehr-frontend-v2/patient-portal
npm install
npm run dev
# Listening on: http://localhost:5174
```

Make sure `src/utils/api.js` points `PATIENT_API` to SYS1:

```
javascript

const PATIENT_API = 'http://<SYS1_IP>:3005'
```

17. Phase 13 — Verification

17.1 Check all peers see the channel

```
bash

# From SYS1 - check peer1 (local)
export FABRIC_CFG_PATH=$HOME/fabric-samples/config
cd ehr-network

CORE_PEER_LOCALMSPID=HospitalMSP \
CORE_PEER_ADDRESS=localhost:9051 \
CORE_PEER_MSPCONFIGPATH=organizations/peerOrganizations/hospital.example.com/users/h
CORE_PEER_TLS_ROOTCERT_FILE=organizations/peerOrganizations/hospital.example.com/pee
peer channel list
# Expected: ehrchannel

# Check peer0 on SYS2 (remote)
CORE_PEER_LOCALMSPID=HospitalMSP \
CORE_PEER_ADDRESS=peer0.hospital.example.com:7051 \
CORE_PEER_MSPCONFIGPATH=organizations/peerOrganizations/hospital.example.com/users/h
CORE_PEER_TLS_ROOTCERT_FILE=organizations/peerOrganizations/hospital.example.com/pee
peer channel list
# Expected: ehrchannel

# Check peer2 on SYS3 (remote)
CORE_PEER_LOCALMSPID=HospitalMSP \
CORE_PEER_ADDRESS=peer2.hospital.example.com:10051 \
CORE_PEER_MSPCONFIGPATH=organizations/peerOrganizations/hospital.example.com/users/h
CORE_PEER_TLS_ROOTCERT_FILE=organizations/peerOrganizations/hospital.example.com/pee
peer channel list
# Expected: ehrchannel
```

17.2 Check gossip discovery

```
bash
```

```
# peer1 (SYS1) should have discovered peer0 (SYS2) via gossip
docker logs peer1.hospital.example.com 2>&1 | grep -i "member\|anchor\|connect" | ta
# Expected: Membership view has changed. peers went online: [peer0.hospital.example.
```

17.3 Check chaincode is committed

```
bash
```

```
CORE_PEER_LOCALMSPID=HospitalMSP \
CORE_PEER_ADDRESS=localhost:9051 \
CORE_PEER_MSPCONFIGPATH=organizations/peerOrganizations/hospital.example.com/users/h
CORE_PEER_TLS_ROOTCERT_FILE=organizations/peerOrganizations/hospital.example.com/pee
peer lifecycle chaincode querycommitted --channelID ehrchannel --name ehr \
--tls --cafile organizations/ordererOrganizations/example.com/orderers/orderer.exa
# Expected: Version: 1.0, Sequence: 1
```

17.4 Check all backend APIs are healthy

```
bash
```

```
# From any machine
curl http://<SYS2_IP>:3001/health      # peer0-api
curl http://<SYS1_IP>:3002/health      # peer1-api
curl http://<SYS3_IP>:3003/health      # peer2-api
curl http://<SYS1_IP>:3004/health      # extorg-api
curl http://<SYS1_IP>:3005/health      # patient-api
curl http://<SYS1_IP>:3006/health      # ipfs-service
```

17.5 Run a login test

```
bash
```

```
# Login as receptionist via peer0-api on SYS2
curl -s -X POST http://<SYS2_IP>:3001/auth/login \
-H "Content-Type: application/json" \
-d '{"username":"receptionist","password":"recept123"}' | jq .

# Expected:
# { "success": true, "data": { "token": "eyJ...", "user": { "role": "receptionist" } }
```

17.6 Check IPFS node

```
bash
```

```
# From SYS1
```

```
curl -s -X POST http://localhost:5001/api/v0/id | jq '.ID, .AgentVersion'
```

```
# Expected: "12D3Koo..." and "kubo/..."
```

17.7 View network status

```
bash
```

```
cd ehr-network
```

```
bash scripts/status.sh
```

18. Environment Variable Reference

peer0-api (SYS2) / peer1-api (SYS1) / peer2-api (SYS3)

```
bash
```

```
PORT=3001 # 3001 / 3002 / 3003
```

```
JWT_SECRET=<min-32-random-chars>
```

```
JWT_EXPIRES_IN=8h
```

```
FABRIC_CHANNEL=ehrchannel
```

```
FABRIC_CHAINCODE=ehr
```

```
FABRIC_BASE_PATH=<abs-path-to-ehr-network> # MUST be absolute
```

```
PEER_ADDRESS=localhost:7051 # 7051 / 9051 / 10051
```

```
IPFS_SERVICE_URL=http://<SYS1_IP>:3006
```

```
IPFS_SERVICE_KEY=change-me-in-production
```

```
LOG_LEVEL=info
```

extorg-api (SYS1)

```
bash
```

```
PORT=3004
```

```
DIAG_PEER_ADDRESS=localhost:8051
```

```
PROV_PEER_ADDRESS=localhost:11051
```

```
IPFS_SERVICE_URL=http://localhost:3006
```

```
FABRIC_BASE_PATH=<abs-path-to-ehr-network>
```

patient-api (SYS1)

```
bash
```

```
PORT=3005
```

```
SQLITE_PATH=./data/patients.db
```

```
FABRIC_BASE_PATH=<abs-path-to-ehr-network>
```

```
PEER_ADDRESS=localhost:7051
```

```
IPFS_SERVICE_URL=http://localhost:3006
```

ipfs-service (SYS1)

```
bash
```

```
PORT=3006
```

```
IPFS_API_URL=http://localhost:5001
```

```
IPFS_GATEWAY_URL=http://localhost:8090
```

```
IPFS_SERVICE_KEY=change-me-in-production
```

```
MAX_BODY_SIZE=5mb
```

19. Port Reference

Port	Service	System
7050	orderer gRPC	SYS1
7053	orderer admin (osnadmin)	SYS1
7054	ca-orderer	SYS1
7055	ca-hospital	SYS1
7056	ca-diagnostics	SYS1
7057	ca-provider	SYS1
7051	peer0.hospital gRPC	SYS2
9051	peer1.hospital gRPC	SYS1
10051	peer2.hospital gRPC	SYS3
8051	peer0.diagnostic gRPC	SYS1
11051	peer0.provider gRPC	SYS1
5001	IPFS Kubo API	SYS1
8090	IPFS HTTP Gateway	SYS1
3001	peer0-api	SYS2
3002	peer1-api	SYS1
3003	peer2-api	SYS3
3004	extorg-api	SYS1
3005	patient-api	SYS1
3006	ipfs-service	SYS1
5173	hospital-portal (dev)	any
5174	patient-portal (dev)	any

20. Troubleshooting

Name or service not known

 for a peer hostname

The peer hostname is not in

/etc/hosts

. Add it manually:

```
bash
```

```
# On SYS1 (if peer0 or peer2 hostnames are missing)
```

```
sudo tee -a /etc/hosts <<EOF
```

```
<SYS2_IP> peer0.hospital.example.com
```

```
<SYS3_IP> peer2.hospital.example.com
```

```
EOF
```

```
# On SYS2 (if peer1, orderer, or provider hostnames are missing)
```

```
sudo tee -a /etc/hosts <<EOF
```

```
<SYS1_IP> orderer.example.com
```

```
<SYS1_IP> peer1.hospital.example.com
```

```
<SYS1_IP> peer0.diagnostic.example.com
```

```
<SYS1_IP> peer0.provider.example.com
```

```
<SYS3_IP> peer2.hospital.example.com
```

```
EOF
```

```
# On SYS3 (if missing entries)
```

```
sudo tee -a /etc/hosts <<EOF
```

```
<SYS1_IP> orderer.example.com
```

```
<SYS1_IP> peer1.hospital.example.com
```

```
<SYS1_IP> peer0.diagnostic.example.com
```

```
<SYS1_IP> peer0.provider.example.com
```

```
<SYS2_IP> peer0.hospital.example.com
```

```
EOF
```

connection refused on peer port

The peer container is not running. SSH to the remote machine and check:

```
bash
```

```
docker ps
```

```
docker logs $(docker ps -aq --filter name=peer0.hospital) --tail 30
```

context deadline exceeded during channel join or chaincode install

The local CLI cannot reach the remote peer. Check in order:

1. Is the peer container running on the remote machine?
2. Is the port open in the firewall?
3. Is the hostname in `/etc/hosts` on SYS1 pointing to the right IP?

```
bash
```

```
nc -zv <SYS2_IP> 7051 # should succeed  
ping peer0.hospital.example.com # should resolve
```

Gossip not connecting (peers in different orgs can't see each other)

Anchor peers may not have been set. Re-run Phase 9. Also check that the peer's `CORE_PEER_GOSSIP_EXTERNALENDPOINT` is set in the docker-compose file to the correct hostname:port — this is what the peer broadcasts to other orgs.

CA enrollment fails (permission error on `organizations/`)

CA containers start as root. Fix ownership after starting CAs:

```
bash
```

```
sudo chown -R $USER:$USER organizations/
```

Chaincode container fails to start on SYS2 / SYS3

Chaincode containers (running inside Docker on the remote machine) need to resolve peer and orderer hostnames. Check `CORE_VM_DOCKER_HOSTCONFIG_EXTRAHOSTS` in the docker-compose file for the peer. It should inject the same host entries that the peer itself has in its `extra_hosts`.

Backend API connects but chaincode calls fail with `access denied`

The role attribute in the certificate doesn't match. Verify:

```
bash
```

```
# Show what role attribute is embedded in the doctor certificate  
openssl x509 -noout -text \  
  -in organizations/peerOrganizations/hospital.example.com/users/doctor/msp/signcert  
  | grep -A2 "1.2.3.4.5.6.7.8.1"  
# Should show: role=doctor
```

If the attribute is missing, re-enroll the identity:

```
bash
```

```
bash scripts/enrollregisteruser.sh \  
  --org hospital --username doctor --password doctor123 \  
  --type client --role doctor
```

IPFS node unreachable from SYS2 / SYS3 backends

Check that port 3006 is accessible from the remote machine:

```
bash

# From SYS2
curl http://<SYS1_IP>:3006/health
```

If blocked, open the port on SYS1:

```
bash

sudo ufw allow from <SYS2_IP> to any port 3006
sudo ufw allow from <SYS3_IP> to any port 3006
```

SQLite error on patient-api startup

The `data/` directory needs to exist:

```
bash

cd ehr-backend-v3/patient-api
mkdir -p data
npm start
```

21. Tear Down and Clean Up

Stop all containers (keep data)

```
bash

# SYS1
cd ehr-network
docker compose -f docker/docker-compose-sys1.yaml stop
docker compose -f docker/docker-compose-ca.yaml stop

# SYS2
docker compose -f ~/Data/fabric-samples/multi-system-test/ehr-network/docker/docker-

# SYS3
docker compose -f ~/Data/fabric-samples/multi-system-test/ehr-network/docker/docker-
```

Full clean (removes all data, channel, chaincode state)

```
bash
```

```
# SYS1 – stop and remove all containers + volumes
```

```
cd ehr-network
```

```
docker compose -f docker/docker-compose-sys1.yaml down -v
```

```
docker compose -f docker/docker-compose-ca.yaml down -v
```

```
# Remove peer volumes (resets ledger)
```

```
docker volume rm docker_peer1.hospital.example.com
```

```
docker volume rm docker_peer0.diagnostic.example.com
```

```
docker volume rm docker_peer0.provider.example.com
```

```
docker volume rm docker_orderer.example.com
```

```
# Remove crypto material (will be regenerated on next enroll)
```

```
rm -rf organizations/
```

```
# SYS2 – stop peer0, remove volume
```

```
ssh <SYS2_USER>@<SYS2_IP> "
```

```
    docker compose -f ~/Data/fabric-samples/multi-system-test/ehr-network/docker/docke
```

```
    docker volume rm docker_peer0.hospital.example.com
```

```
"
```

```
# SYS3 – stop peer2, remove volume
```

```
ssh <SYS3_USER>@<SYS3_IP> "
```

```
    docker compose -f ~/Data/fabric-samples/multi-system-test/ehr-network/docker/docke
```

```
    docker volume rm docker_peer2.hospital.example.com
```

```
"
```

After a full clean, restart from **Phase 3** (Start CAs and Enroll).

Quick Reference — Startup Order

Phase 3 → Start CAs + Enroll (SYS1)

Phase 4 → Distribute (SYS1 → SYS2, SYS3)

Phase 5 → Start peer0 (SYS2)

Phase 6 → Start peer2 (SYS3)

Phase 7 → Start orderer + peer1 + diagnostic + provider + IPFS (SYS1)

Phase 8 → Create channel, join all 5 peers (SYS1)

Phase 9 → Set anchor peers (SYS1)

Phase 10 → Deploy chaincode (SYS1)

Phase 11 → Start backends (SYS2: peer0-api, SYS3: peer2-api, SYS1: rest)

Phase 12 → Start frontends

Phase 13 → Verify everything

*EHR System · Hyperledger Fabric 2.5 + IPFS · NIT Warangal MTech Minor Project Hospital
Organization: Avijit Ram (25CSM2R05), Jenish Shiroya (25CSM2R10), Rahul Yadav
(25CSM2R16)*